EX27 · CLASS_17_VECTORS · 03 / EXERCISE GUIDE

EX27: Cross product and moments

First vector algebra, then force moments. You write the Python program.

Learning sequence
01 Math theory → 02 Python reference → 03 Exercise → 04 Pseudocode → Programming.

Prerequisites: EX25 and EX26, NumPy arrays, Euclidean norm and dot product.

1. Goals and constraints

Compute 3D cross products, check direction and magnitude, then calculate moments about a reference point.

Requirements: one script using NumPy and previously taught functions. No JSON, `pytest`, classes or new modules. Custom functions are not required.

2. Part A — Solved algebraic reference

Check the Cartesian unit-vector relations by hand. This example is fully solved as a reference.

$$\mathbf{a} = (1, 0, 0), \quad \mathbf{b} = (0, 1, 0)$$
$$\mathbf{a} \times \mathbf{b} = (0, 0, 1)$$
$$\mathbf{b} \times \mathbf{a} = (0, 0, -1)$$

Both results are vectors of magnitude 1 but opposite direction. Verify them with `np.cross`.

3. Part A — Your algebra task

Now apply the same approach to two vectors with three non-axis-aligned components. Write the code yourself.

$$\mathbf{u} = (2, -1, 3), \quad \mathbf{v} = (1, 4, -2)$$

1. Create the two NumPy arrays with three components.

2. Compute both **u** × **v** and **v** × **u**.

3. Compute the Euclidean norm of the first cross product.

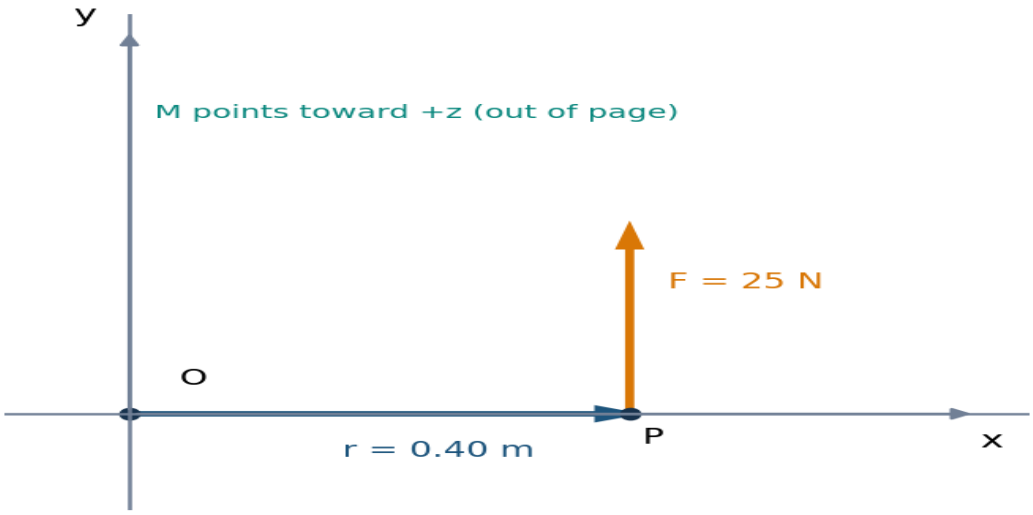
4. Check the result is perpendicular to both original vectors by using dot products.

5. Print the results and compare with the checkpoints.

Check	Expected
u × v	(−10, 7, 9)
v × u	(10, −7, −9)
Norm of u × v	√230 ≈ 15.1658
u · (u × v)	0
v · (u × v)	0

4. Part B — Moment of a planar force

A 25 N force toward +y acts at the end of a horizontal 0.40 m lever arm. Compute its moment about origin O. Although the geometry is planar, use 3D arrays to capture the moment direction.



Problem diagram. The position vector points from O to the force application point.

$$\mathbf{r}_{OP} = (0.40, 0, 0)\ \text{m}, \quad \mathbf{F} = (0, 25, 0)\ \text{N}$$

$$\mathbf{M}_O = \mathbf{r}_{OP} \times \mathbf{F}$$

1. Represent both vectors with NumPy.

2. Compute the moment vector, keeping the correct argument order.

3. Compute its magnitude.

4. Explain whether the direction is +z or −z, and why.

Check B
Moment vector: (0, 0, 10) N·m. Magnitude: 10 N·m. Direction: +z, out of the xy plane.

5. Part C — Independent 3D case

A second force acts at a position offset from the main axes. No new methods are needed: apply what you have learned.

$$\mathbf{r} = (0.30, 0.20, 0)\ \text{m}, \quad \mathbf{F} = (10, -5, 20)\ \text{N}$$

1. Compute the moment vector about the origin.

2. Compute its norm and report the result in N·m.

3. Manually check one component and explain its sign.

> **Check C**
> Moment vector: (4, −6, −3.5) N·m. Approximate norm: 8.016 N·m.

6. Implementation requirements and folder structure

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        └── class_17_vectors/
            ├── EX25_vectors_and_forces.py
            ├── EX26_dot_product.py
            ├── EX27_cross_product.py   <--- NEW
            └── docs/
                ├── ES/
                └── EN/
```

Write your work in `EX27_cross_product.py`. Separate A, B and C with short comments. You may write pseudocode on paper before implementing it.

7. Interpretation questions

Answer briefly in comments or when submitting your work:

• Why is the cross product a vector instead of a scalar?

• What changes when we reverse the vector order?

• Why does a planar-force problem produce a z moment?

• Why are moments measured in N·m?

8. Code hints — at the end

The following snippets show syntax but **do not solve** your tasks. You decide the variables, operation order and output checks.

```
import numpy as np

vector = np.array([1.0, 0.0, 0.0])
other = np.array([0.0, 1.0, 0.0])

normal = np.cross(vector, other)
size = np.linalg.norm(normal)
check = np.dot(vector, normal)
```